Name: Arya Bailur
Roll No: 08
Class: T1
Subject: Agile Software Development and DevOps Lab

Experiment 4

Aim: To understand continuous integration, Jenkins installation

This experiment maps to LO1,LO3.
LO1:Apply Agile methodologies and use JIRA for sprint planning, task tracking and integrating DevOp workflows
LO3:Build and Manage automated CI/CD Pipeline for application development, testing, packaging, and deployment.

Theory:

Jenkins is an open-source automation server used for implementing Continuous Integration (CI) and Continuous Delivery/Deployment (CD). It automates tasks such as building code, running tests, generating reports, packaging applications, and deploying software. Jenkins supports integration with tools like Git, Maven, Gradle, JUnit, and cloud platforms through its large plugin ecosystem. It is developed in Java and runs on the JVM, providing a web-based interface to manage projects, pipelines, users, and plugins.

A Jenkins Pipeline is a set of automated steps that defines the software delivery process. It is written in a file called Jenkinsfile, which stores pipeline instructions as code. A typical pipeline includes stages like:

- Build – Compiles source code and creates the application.
- Test – Runs automated tests to check quality and functionality.
- Deploy – Releases the application to servers or production.

Advantages of Jenkins Pipeline

- Automates software delivery.
- Reduces manual errors.
- Speeds up development and testing.
- Provides better visibility of the build process.
- Supports continuous integration and deployment.
- Allows pipeline configuration to be stored and shared as code.

Types of Jenkins Pipelines

1. Declarative Pipeline
   - Uses the pipeline {} block.
   - Has a simple, fixed structure.
   - Easier to read and maintain.
   - Suitable for beginners.

Example:

```
pipeline {
    agent any
    stages {
        stage('Build') {
            steps {
                echo 'Building application'
            }
        }
    }
}
```

2. Scripted Pipeline
   - Written using Groovy scripting.
   - Provides more flexibility and control.
   - Suitable for complex workflows.
   - Requires more programming knowledge.

Example:

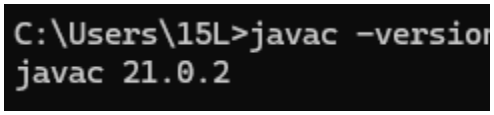
```
node {
    stage('Build') {
        echo 'Building application'
```

```
    }
}
```

Jenkins plays an important role in DevOps by automating the software development lifecycle, improving efficiency, reducing errors, and enabling faster and more reliable software delivery.

Implementation

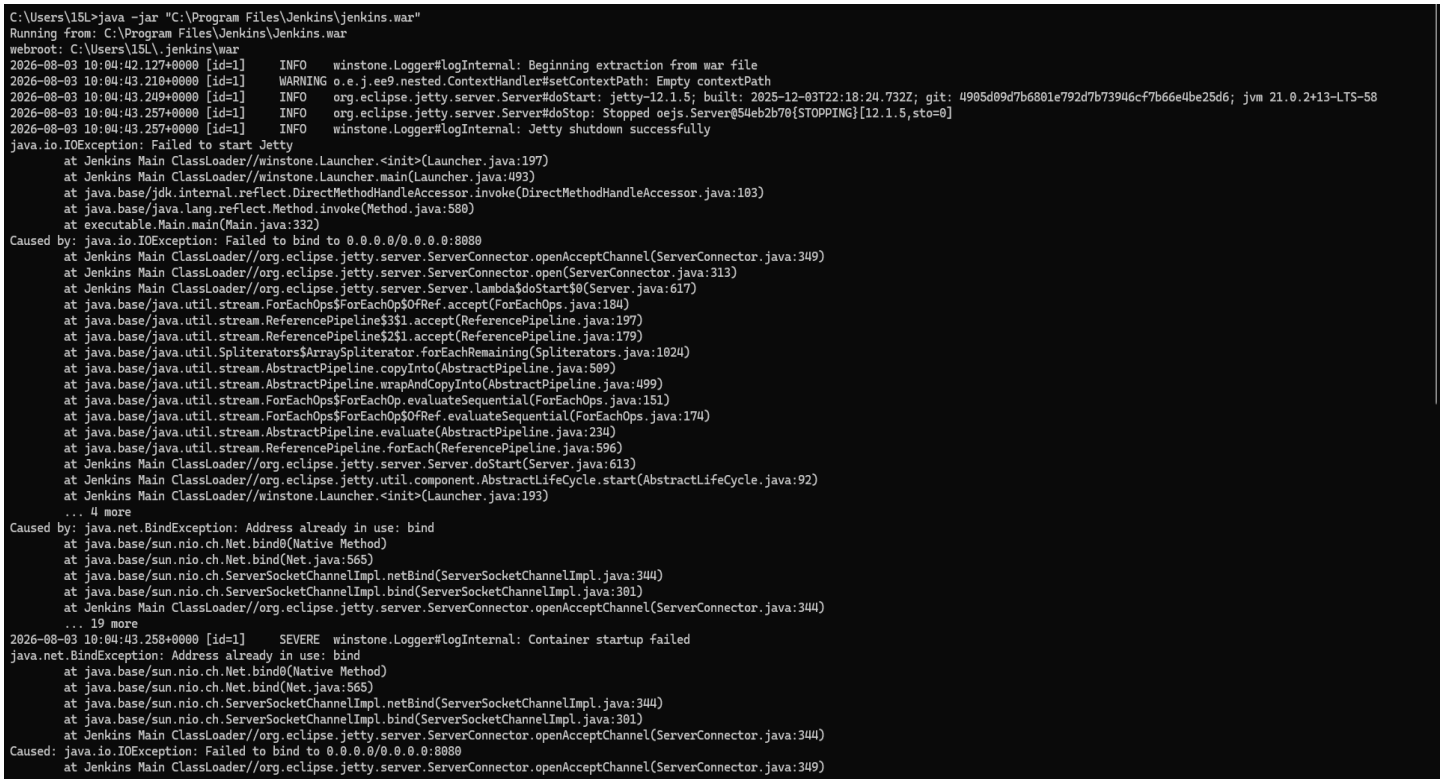
1. Check if JDK (Java Development Kit) is installed.



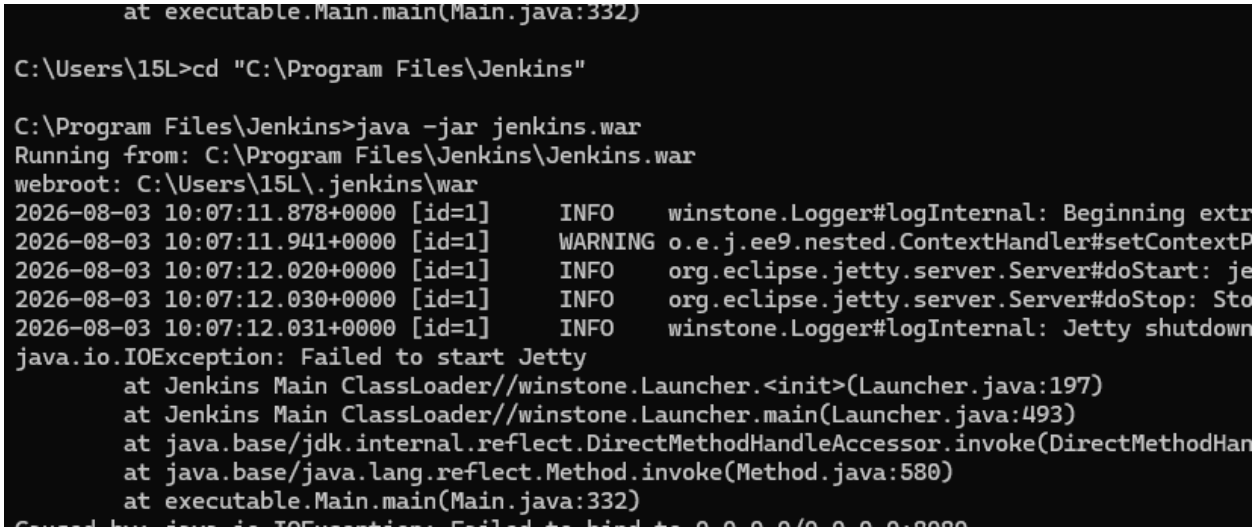
2. Download Jenkins WAR file from official website or packet manager



3. Launch command prompt on your system and give the following command

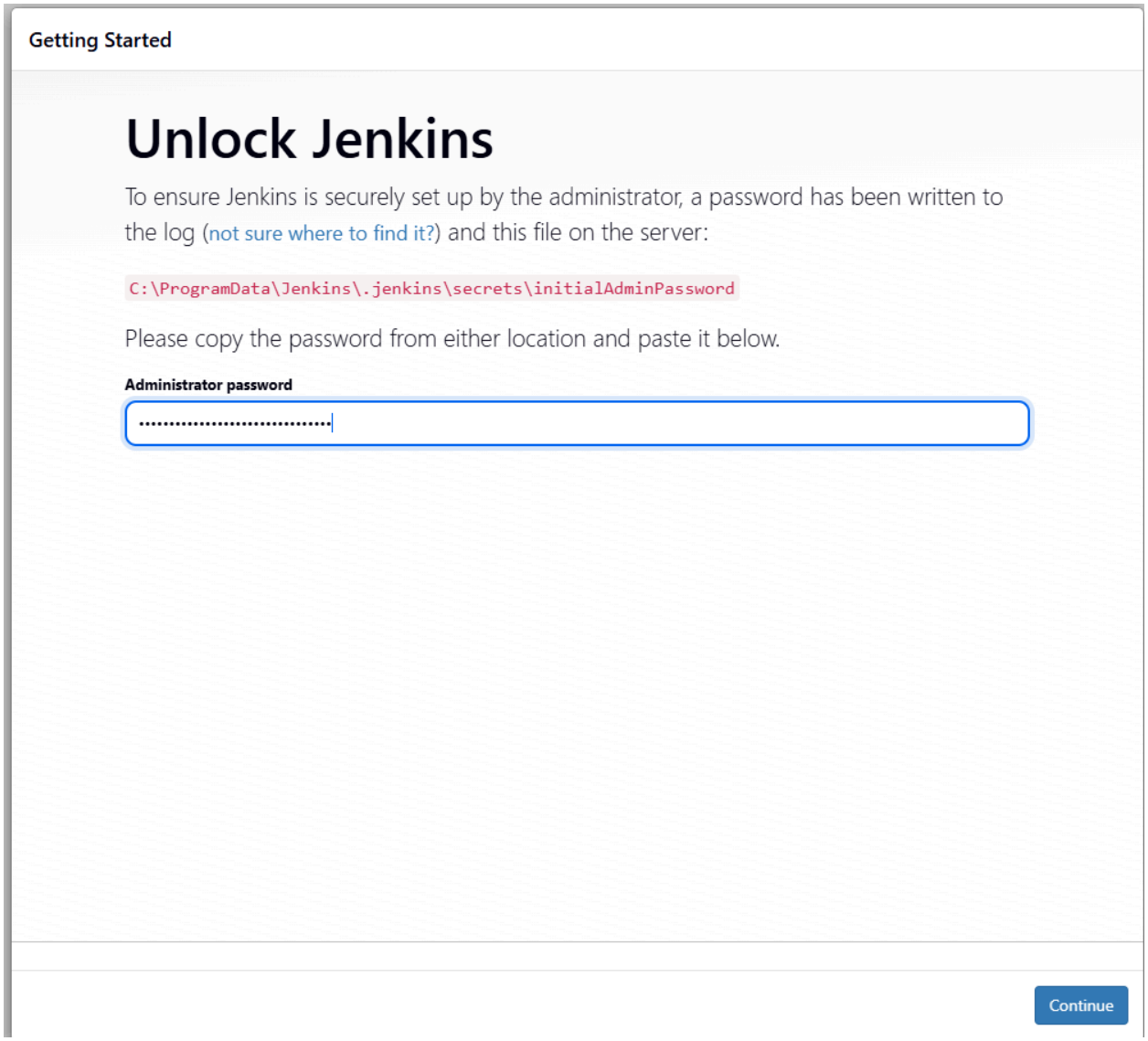


4. Navigate to the folder

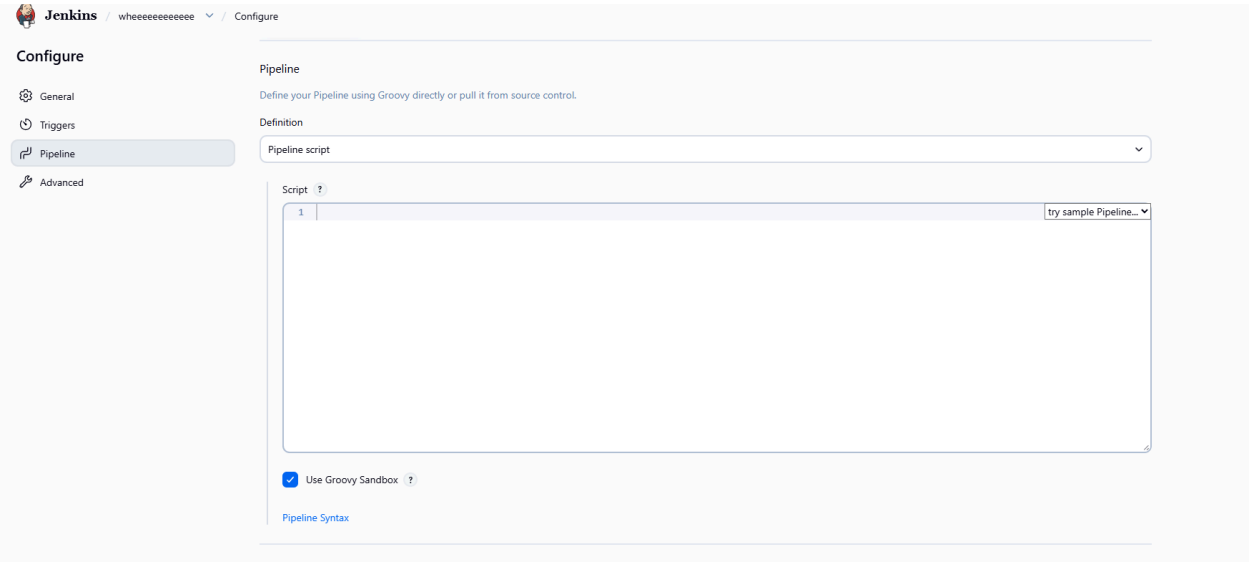


5. Start Jenkins: Run the command C:\Program Files\Jenkins>java -jar jenkins.war to start Jenkins server.

6. Access Web UI: open localhost 8090



7. Open jenkins and pipeline

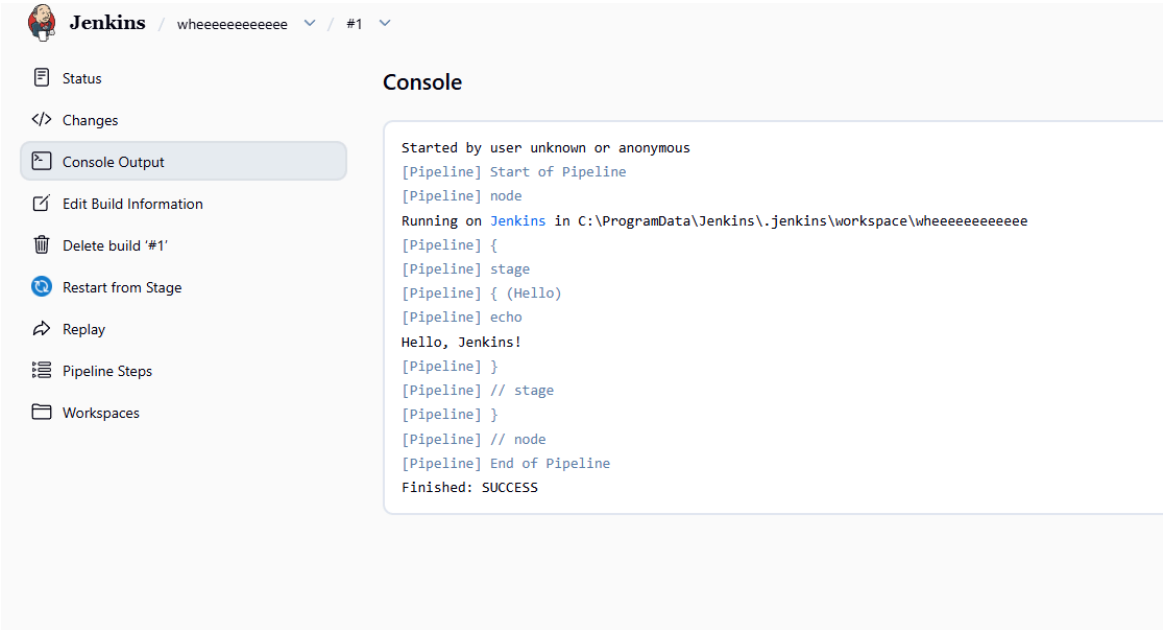


Code:

```
pipeline {
    agent any

    stages {
        stage('Hello') {
            steps {
                echo 'Hello, Jenkins!'
            }
        }
    }
}
```

Output:



Conclusion:

Jenkins Pipeline automates the software development process by integrating stages such as build, testing, and deployment to support Continuous Integration and Continuous Delivery. In this experiment, Jenkins was successfully installed and configured by verifying the JDK, launching Jenkins using the `jenkins.war` file, and accessing the setup page through a web browser. A Declarative Pipeline was then created and executed, demonstrating how Jenkins automates development tasks and enhances the efficiency, consistency, and reliability of the Continuous Integration and Continuous Delivery process.